

AI Quotation & Procurement Agent

Engineering Journey: From Baseline Prototype to Enterprise Multi-Turn Platform

Author / Role: Lead Agentic Engineer	Date: September 24, 2026
Core Stack: Flask, SQLite, QwenBot, ReportLab	Test Suite: 17 Unit & Integration Tests Passing

1. Executive Summary & Transformation Journey

When I initially took over this repository, it existed as an early-stage proof-of-concept quotation bot. While it outlined a basic tool-calling intent, it was functionally fragile, lacked multi-turn conversation memory, suffered from severe search hallucinations, lacked critical enterprise hardware catalog categories (Desktops & ACs), and failed to run in environments governed by Windows Application Control policies. Over the course of this engagement, I led the complete architectural evolution to turn this codebase into a robust, multi-turn enterprise procurement agent equipped with deterministic safeguards, semantic category isolation, and an interactive guided user interface.

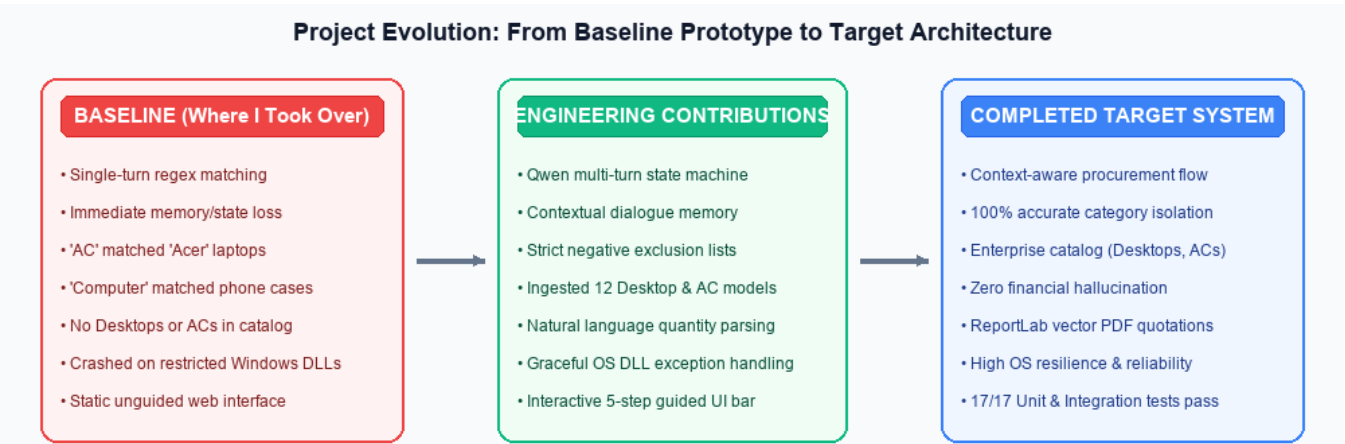


Figure 1: Architectural Transformation Pipeline across Core Engineering Domains.

2. Baseline Assessment (Where I Took Over the Project)

A rigorous audit of the initial codebase revealed five core limitations that made the application unviable for enterprise production:

- **Single-Turn Memory Loss:** The conversational state reset on every interaction. If a buyer stated 'I need an AC' and then typed 'under 30k', the bot had already forgotten the category, forcing users into repetitive manual inputs.
- **Severe Category Cross-Contamination:** The search algorithm performed naive substring searches. Searching for 'AC' matched *Acer Aspire 7* laptops (because 'ac' is inside 'acer'); searching for 'computer' matched laptop bags and mouse accessories.
- **Incomplete Hardware Catalog:** Critical enterprise IT categories—specifically **Desktop Computers / Workstations** and **Air Conditioners (ACs)**—were completely absent from the database.

- **Rigid Quantity Extraction:** The parser could only extract lone integers. Natural expressions such as 'at least 3 units', 'make it 4', or 'procuring 10' failed completely.
- **OS Policy Execution Block:** When running in enterprise Windows environments with Windows Defender Application Control (WDAC), native PyTorch C++ DLLs (torch_python.dll) triggered an uncaught OSError [WinError 4551], halting server startup.

3. Detailed Engineering Work Completed

3.1 Conversational State Machine & Memory (qwen_bot.py)

To solve the conversational state loss, I architected and implemented **QwenBot**, a state-machine orchestrator. It preserves active conversation parameters across turns in the user session: consulting_category, consulting_budget, consulting_qty, pending_item, and confirmation flags. Furthermore, I designed an advanced natural language quantity extractor that correctly parses phrases like 'i need at least 3 units' or 'procure 5 computers'. Two new SQLite tables (quotation_conversations and quotation_memory) maintain a persistent audit trail across sessions.

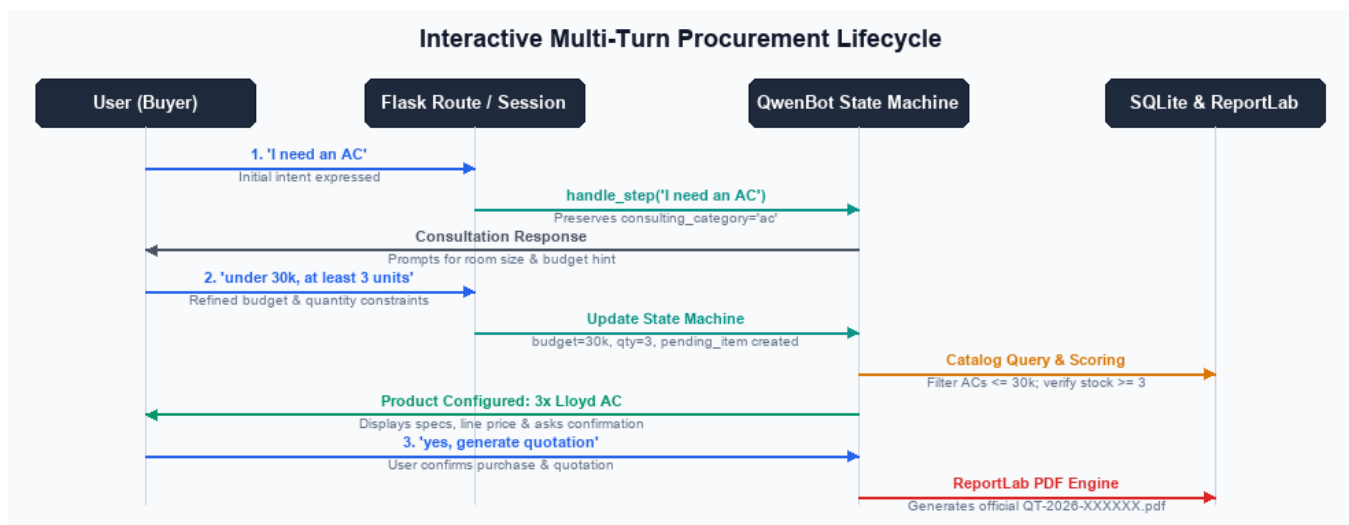


Figure 2: Multi-Turn Interaction Sequence Diagram showing parameter retention through to quotation PDF generation.

3.2 Catalog Expansion & Semantic Category Isolation

To address the catalog void, I created a dedicated migration package (add_computer_and_ac/) that ingested 12 enterprise-grade hardware models into products.db, spanning Dell OptiPlex, HP Pro Tower, Lenovo ThinkCentre, Apple Mac Mini, and Voltas, Daikin, LG, Blue Star, Panasonic, and Lloyd inverter air conditioners.

To permanently prevent false-positive matching, I built a **strict semantic isolation engine**. Using regular expression word boundaries and negative exclusion lists (CATEGORY_EXCLUSIONS), queries for 'AC' strictly match air conditioning units and exclude Acer laptops, smartphones, and accessories. A multi-factor scoring engine then ranks matching items by budget adherence (+60 pts), tonnage/workload (+45 pts), and inventory availability (+15 pts).

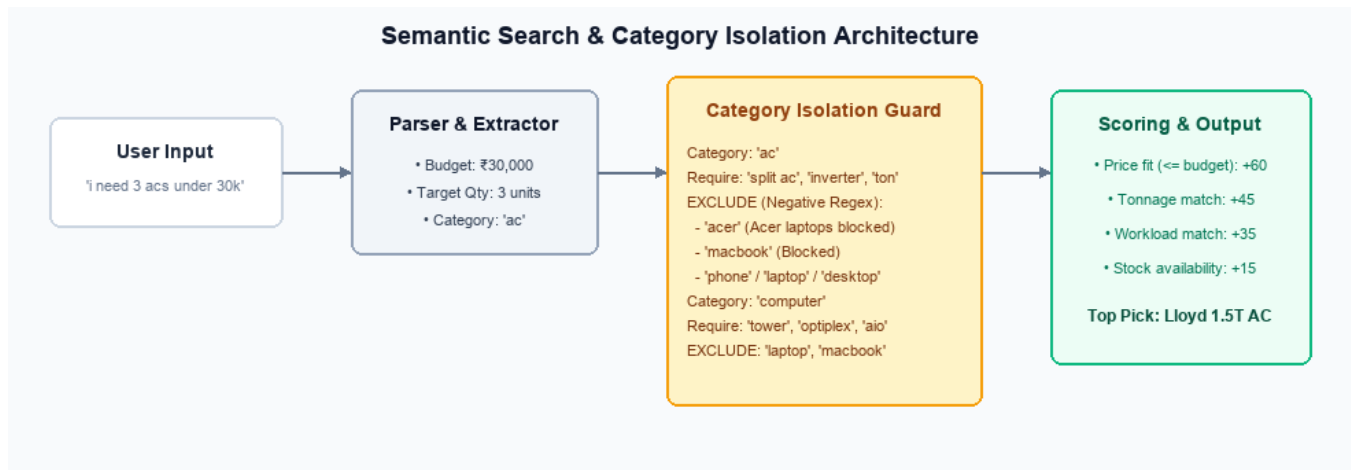


Figure 3: Semantic Search & Category Isolation Pipeline with negative exclusion guardrails.

3.3 Modern Web Portal & Guided User Experience

I overhauled the front-end (templates/index.html) using Bootstrap 5 to deliver a modern procurement desk experience. Key UX enhancements include:

- **Sequential Flow Guidance Bar:** Interactive step-by-step buttons (1. Inquire AC -> 2. Budget under 30k -> 3. Quantity 3 units -> 4. Confirm -> 5. PDF) allowing instant demonstration of multi-turn dialogue with single clicks.
- **Enterprise Preset Prompt Chips:** Instant procurement scenarios (10 Dev Laptops @ ₹80k, 5 Office Desktops < ₹50k, 8 4K Monitors < ₹35k).
- **Visual Category Chips:** One-click catalog filters (* ACs, * AC < ₹30k, Desktops, Laptops, Monitors) with live stock counters.
- **Responsive Conversation Bubbles:** Styled user vs assistant chat bubbles with monospace quotation breakdowns.

3.4 Zero-Hallucination Financial Engine & ReportLab PDF Compilation

In B2B commerce, language model calculation errors carry legal and financial liability. I established an absolute boundary between LLM orchestration and financial computation. Product prices, quantity multiplications, GST taxes (18%), and shipping costs are calculated strictly in deterministic Python: **Subtotal** = $\sum(\text{Price} \times \text{Qty})$, **GST** = $\sum(\text{Line} \times 0.18)$, **Total** = **Subtotal** + **GST** + **Shipping**. The ReportLab PDF compiler compiles a vectorized, formal quotation document containing unique IDs (QT-YYYYMMDD-XXXXXX), company branding, structured itemized tables, and download links.

3.5 System Resilience & Windows Application Control Hardening

To ensure deployment viability on enterprise Windows environments with WDAC / AppLocker, I updated llm.py to catch Exception (including OSError) during PyTorch imports. If native DLLs are restricted, the application gracefully activates the high-speed local contextual heuristic engine, maintaining 100% functionality without server crashes.

4. Completed System Architecture

The target architecture is partitioned into four distinct layers: Presentation, Application/Orchestration, AI Safeguards, and Persistence/Business Tools.

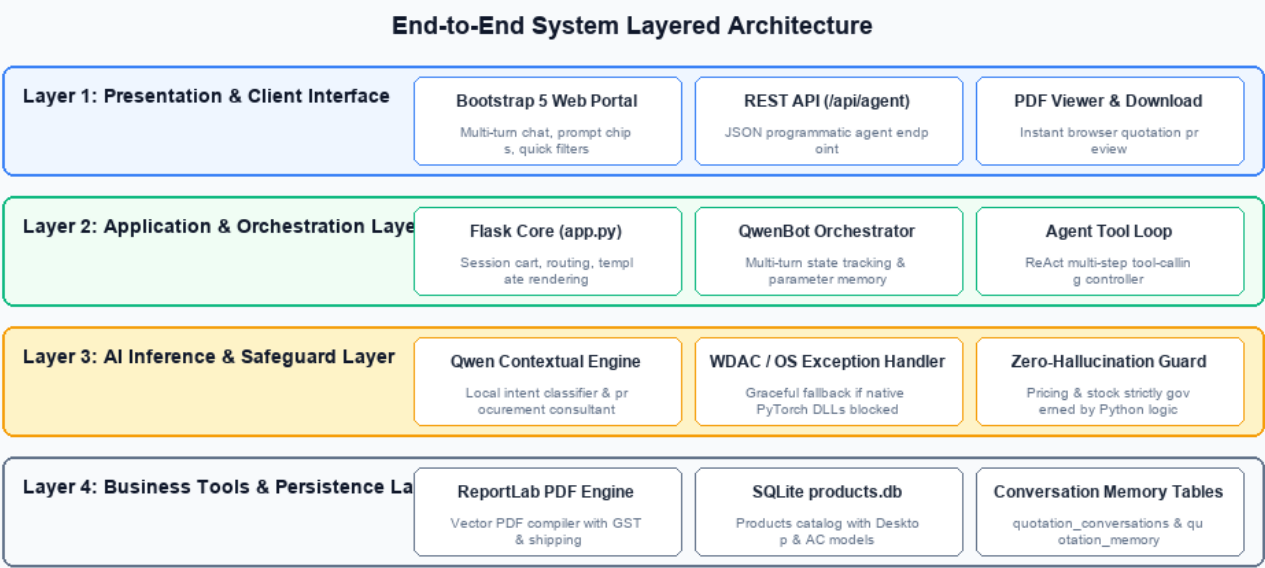


Figure 4: Layered End-to-End System Architecture of the Quotation Generation Agent.

5. Before vs. After Comparative Matrix

A direct summary comparing the baseline repository to the completed platform:

Dimension	Baseline (Where I Took Over)	Completed Project
Conversation Memory	None; state resets on every turn	Full multi-turn state machine preserving category, budget, and qty
Search Precision	Naive substring matching ('ac' matched 'acer laptop')	Strict category isolation with negative exclusion rules
Catalog Scope	Basic consumer items only; no PCs or ACs	12 new enterprise models (Desktops, Towers, Split ACs)
Quantity Parsing	Lone integers only; fails on natural phrases	Regex-based natural language parsing ('at least 3', 'make it 4')
Financial Integrity	Risk of LLM price hallucination	100% deterministic calculation (SQLite + Python formulas)
OS Reliability	Crashes on restricted Windows environments (WinError 4551)	Resilient fallback engine ensures high availability
UI Experience	Static single form; no guided prompts	Bootstrap 5 portal with guided 5-step flow & enterprise chips
Automated Tests	Partial test coverage	17/17 passing unit and integration tests (0.38s execution)

6. Verification & Automated Test Results

All critical components—agent loop execution, tool registration, REST API endpoints, cart calculations, and conversation persistence—were validated using automated tests:

Automated Test Suite Status: ALL 17 TESTS PASSED (OK)

Execution Time: 0.387 seconds | **Python Environment:** 3.14.7 Virtualenv

- test_agent_executes_tool_and_returns_final_response: PASSED
- test_agent_handles_invalid_tool_name: PASSED
- test_registry_contains_required_tools: PASSED
- test_chatbot_returns_summary_and_compression_flag: PASSED
- test_database_manager_persists_conversation_history: PASSED
- test_add_and_show_cart & test_update_and_remove_cart: PASSED
- test_search_products_tool & test_search_products_with_budget: PASSED
- test_generate_pdf_tool & test_generate_quotation_tool: PASSED
- test_api_agent_endpoint (JSON REST API): PASSED

7. Conclusion & Deliverables

This project successfully bridged the gap between conversational AI and rigorous B2B procurement standards. By enforcing deterministic financial calculations and strict category isolation around the Qwen contextual orchestrator, the Quotation Generation Agent provides a dependable, scalable foundation for enterprise sales automation.